Low Level Design

On

Entertainment Content Generator

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Maitry Banduke	EN22CS301571
2.	Mohd Quasim	EN22CS301603
3.	Mrunali Kamerikar	EN22CS301618
4.	Muskan Asija	EN22CS301623

Group Name: G3 D6

Project Number: GAI -14

Industry Mentor Name:

University Mentor Name: Prof. Ajaj Khan

Academic Year: 2025-2026

Table of the Content

1. **Introduction**
   - 1.1 Scope of the document
   - 1.2 Intended Audience
   - 1.3 System Overview

2. **System Design**
   - 2.1 Application Design
   - 2.2 Process Flow
   - 2.3 Information Flow
   - 2.4 Components Design
   - 2.5 Key Design Considerations
   - 2.6 API Catalogue

3. **References**

1. Introduction

The Entertainment Content Generator is an AI-based application that helps users create content such as scripts, dialogues, and story ideas easily and efficiently. It uses technologies like Large Language Models (LLMs) to understand user input and generate creative and well-structured content from simple prompts. The system also uses a vector database to store embeddings and retrieve similar content, ensuring consistency in tone and quality. The backend is developed in Python and integrated with AI APIs for smooth content generation. Overall, the system saves time, reduces manual effort, and helps content creators produce high-quality outputs quickly and effectively.

1.1 Scope of the document

This document provides a detailed Low-Level Design (LLD) of the Entertainment Content Generator system, explaining how the system is built and how its components function internally. It describes the structure, behavior, and interaction of key modules such as the frontend, backend, and integration with Large Language Models (LLMs) used for content generation. The document also explains the complete workflow of prompt processing, from user input to the generation of structured and creative content. It highlights the use of a vector database, including how embeddings are stored and how semantic similarity search is used to retrieve relevant context, improving the quality and consistency of outputs.

Additionally, the document outlines the API design, including endpoints, request-response formats, and communication between system components. It explains the data flow across modules, showing how information moves between the frontend, backend, LLM services, and database systems. Important aspects such as error handling, validation, and performance optimization are also included to ensure the system is reliable, scalable, and efficient. Overall, this document serves as a comprehensive guide to the system's internal working and supports its development and maintenance.

1.2 Intended Audience

Developers:

This document is useful for developers as it provides a clear understanding of the system's implementation. It explains how the frontend, backend, APIs, and database are connected, along with the flow of data between modules.

Developers can use this information for coding, testing, debugging, maintaining the application, and understanding system behaviour for future enhancements.

AI Engineers:
AI engineers can use this document to understand how Large Language Models (LLMs) are integrated into the system and how prompt engineering techniques are applied for content generation. It also explains the use of vector databases and Retrieval-Augmented Generation (RAG), helping improve the relevance, accuracy, and quality of outputs.

System Designers:
This document helps system designers analyze the overall architecture and design of the system. It provides insights into component structure, data flow, scalability, and performance considerations, supporting better design decisions and future optimizations.

Academic Evaluators:
Academic evaluators can refer to this document to assess the correctness, completeness, and technical depth of the system design. It helps them evaluate whether the project follows proper design principles and meets academic standards.

Project Stakeholders:
Project stakeholders, such as managers or clients, can use this document to understand the system's functionality, features, and purpose. It helps them evaluate whether the system meets project goals and supports informed decision-making.

1.3 System overview

The Entertainment Content Generator system follows a Generative AI (GenAI)-based modular architecture that combines user input, intelligent processing, and AI-based content generation. It is designed to convert simple user prompts into structured and creative outputs such as scripts, dialogues, and story ideas. The system integrates components like the frontend, backend, Large Language Models (LLMs), and a vector database to ensure high-quality and context-aware results.

The workflow starts when the user provides a prompt, which is sent to the backend for processing.

The backend enhances the input using prompt engineering techniques and retrieves relevant context from the vector database using semantic similarity search. This enriched prompt is then sent to the LLM API, which generates structured content. The output is formatted and returned to the user through the frontend.

2. System Design

2.1 Application Design

The Entertainment Content Generator is designed using a modular Generative AI (GenAI) architecture that combines user interaction, backend processing, and AI-based content generation. It converts simple user prompts into high-quality outputs such as scripts, dialogues, and story ideas while maintaining consistency and relevance. The frontend provides a user-friendly interface where users can enter prompts and view the generated content in a structured format, and the input is then sent to the backend for processing.

The backend, developed in Python, acts as the central controller of the system. It processes user input and enhances it using prompt engineering techniques. The system follows a Retrieval-Augmented Generation (RAG) approach, where a vector database stores embeddings and retrieves relevant context using semantic similarity search, which is combined with the original prompt to improve output quality.

The enriched prompt is then sent to the LLM API, which generates structured and context-aware content. The backend manages the complete workflow, including pre-processing, context retrieval, API communication, and formatting of the final output before sending it back to the frontend.

2.2 Process Flow

The process flow explains how the system generates content from user input in a structured way. First, the user enters a prompt in the frontend interface, such as "Generate a thriller script scene." This input is sent to the backend through an API request, where it is processed and refined using prompt engineering techniques. The system then converts the input into embeddings, representing the data in numerical form for further processing.

These embeddings are used to query the vector database, which retrieves relevant contextual information using semantic similarity search. The retrieved context is combined with the original prompt to create an enriched input, improving the relevance and quality of the generated content.

Finally, the enriched prompt is sent to the LLM API, which generates structured and creative content such as scripts or dialogues. The backend formats the output and sends it back to the frontend, where it is displayed to the user, ensuring a smooth and high-quality content generation experience.

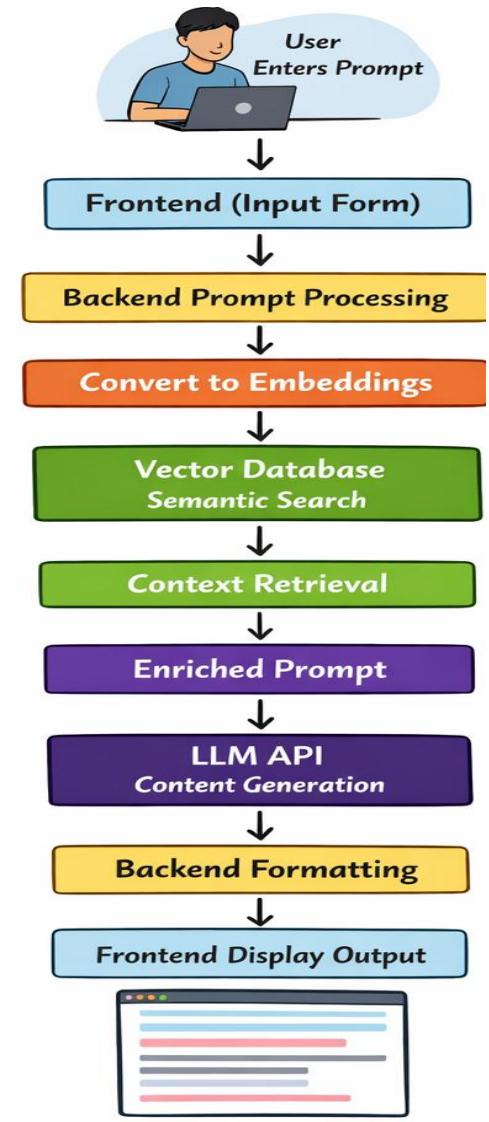

Figure 1: Process Flows

2.3Information Flow

The information flow describes how data moves through the system from user input to final output in a structured manner. The user enters a prompt through the frontend interface, which is sent to the backend using REST APIs for processing. The backend converts the input into embeddings using an embedding model, allowing the system to represent the data in numerical form for efficient processing.

These embeddings are used to query the vector database, which performs semantic similarity search to retrieve relevant context. The backend combines this context with the original input to create an enriched prompt, improving the quality and relevance of the output. This enriched prompt is then sent to the LLM API, which generates structured and creative content such as scripts or dialogues. The generated output is returned to the backend and formatted properly.

Finally, the formatted response is sent back to the frontend and displayed to the user. This flow ensures smooth communication between components and enables accurate, context-aware, and high-quality content generation.

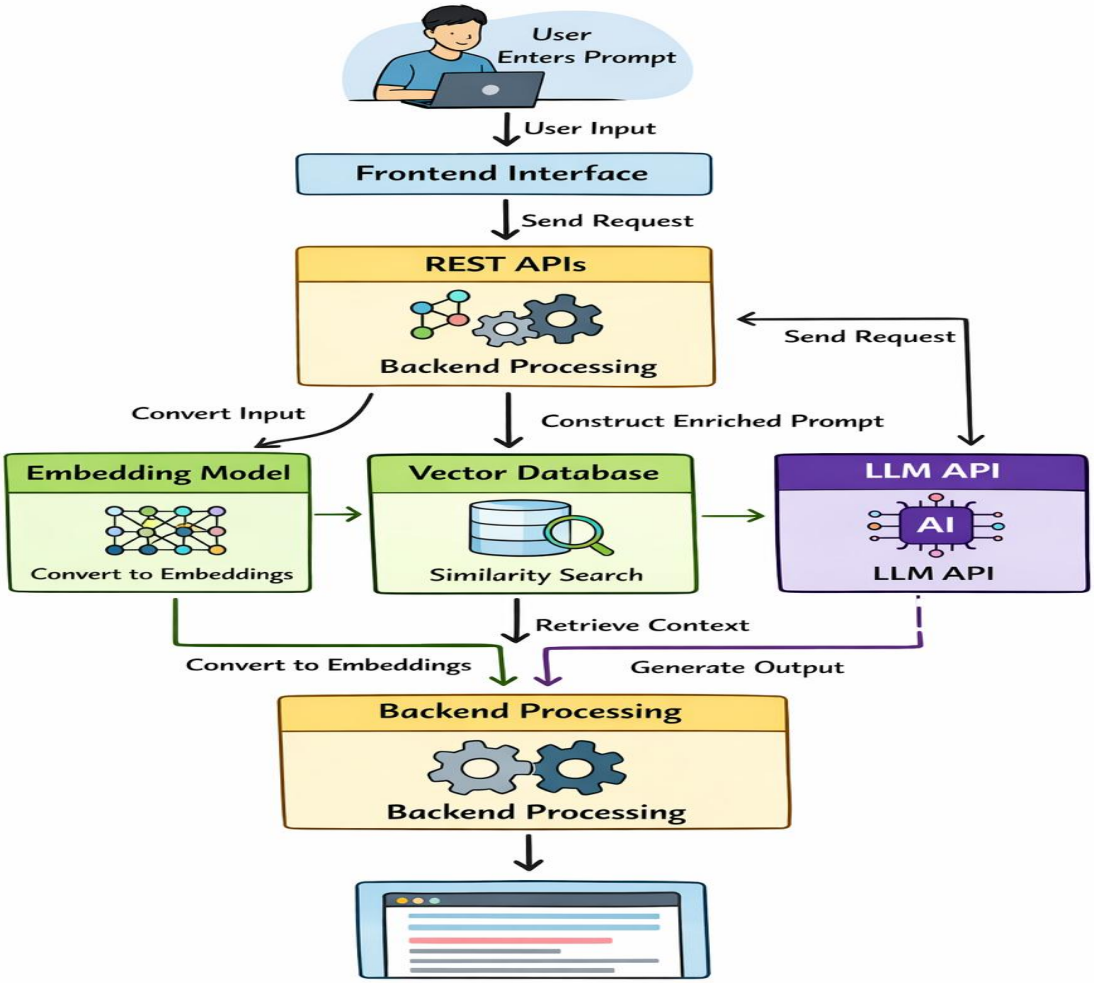

Figure 2 : Information Flow

2.4 Components Design

The system consists of several important components that work together to provide efficient and intelligent content generation. The frontend acts as the user interface, allowing users to enter prompts and view results in a clear and organized format, while also handling basic input validation. The backend, developed in Python, is the core component that manages system logic, processes user inputs, applies prompt engineering, and handles API communication with the vector database and the LLM API. The LLM API generates creative and structured content based on the enriched prompts.

The vector database stores embeddings of documents, templates, and previous content, and retrieves relevant context using semantic similarity search. The embedding model converts text into vector form for efficient matching, while the prompt engineering module enhances input with structure and context to ensure high-quality outputs. Together, these components enable accurate, consistent, and context-aware content generation.

2.5 Key Design Considerations

The design of the Entertainment Content Generator system is based on several important considerations to ensure efficiency, scalability, and high-quality content generation.

1. Scalability:
   The system is designed using a modular architecture, allowing independent scaling of components such as the backend, vector database, and LLM services. This ensures that the system can handle an increasing number of users and requests without performance degradation.

2. Performance Optimization:
   Efficient processing is achieved through the use of vector embeddings and semantic similarity search, which enables faster retrieval of relevant context. Optimized API calls to the LLM reduce latency and improve response time.

3. Accuracy and Quality of Output:
   The system uses Retrieval-Augmented Generation (RAG) to enhance the quality and relevance of generated content. By combining user input with context retrieved from the vector database, the system produces more accurate, consistent, and meaningful outputs.

4. Reliability:
   The system includes proper error handling and fallback mechanisms to manage failures in API calls or data retrieval. This ensures continuous operation and a smooth user experience even in case of partial system failures.

5. Maintainability:
   The system is designed with a clear separation of concerns between frontend, backend, and AI components. This makes it easier to update, debug, and extend the system without affecting other modules.

6. Security:
   Sensitive information such as API keys is securely managed using environment variables. Input validation is implemented to prevent misuse and ensure safe interaction with the system.

2.6 API Catalogue

The system provides a set of REST APIs that enable smooth and efficient communication between the frontend, backend, and AI components. These APIs act as a bridge between different modules, ensuring that user requests are properly processed and responses are delivered in a structured and reliable manner. They help maintain a clear flow of data across the system and support seamless integration between various services.

The Generate Content API is the primary endpoint used to generate content based on user input. It accepts parameters such as the prompt and an optional genre, and returns the generated content in text format.

The Context Retrieval API is responsible for fetching relevant contextual information from the vector database by taking a query as input and returning a list of similar documents. The Embedding API plays an important role by converting input text into vector representations, which are used for similarity search and enable the Retrieval-Augmented Generation (RAG) process, improving the relevance of the output.

3. References

1. OpenAI Documentation – https://platform.openai.com/docs
2. Lewis et al., Retrieval-Augmented Generation (RAG), 2020 – https://arxiv.org/abs/2005.11401
3. Pinecone Vector Database Documentation – https://docs.pinecone.io
4. Prompt Engineering Guide – https://www.promptingguide.ai
5. REST API Tutorial – https://restfulapi.net
6. GitHub Repository – https://github.com/Mrunali-Kamerikar/Entertainment-Content-Generator